



DBMS HW 3

謝侑哲

112550069

1. Screenshot of Footers

The image displays three screenshots of a web application's footer area, each showing a different form or message. The first screenshot shows the 'Login' form with fields for 'Username' and 'Password', a 'Log In' button, and a link to 'Sign Up'. The second screenshot shows a 'Welcome, admin!' message with a 'Log Out' button. The third screenshot shows the 'Sign Up' form with fields for 'Username' and 'Password', a 'Sign Up' button, and a link to 'Back to Login'. Each screenshot includes a copyright notice at the bottom: '© 2024 NYCU DB HW3 112550069 謝侑哲. All rights reserved.'

Login

Username:
Enter your username

Password:
Enter your password

Log In

Don't have an account? [Sign Up](#)

© 2024 NYCU DB HW3 112550069 謝侑哲. All rights reserved.

Welcome, admin!

We're glad to have you back!

Log Out

© 2024 NYCU DB HW3 112550069 謝侑哲. All rights reserved.

Sign Up

Username:
guest

Password:
...

Sign Up

Already have an account? [Back to Login](#)

© 2024 NYCU DB HW3 112550069 謝侑哲. All rights reserved.

2. Screenshot and Explanation of Login Function

```

# Login Page
@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form['username']
        password = request.form['password']

        # TODO # 4: Hash the password using SHA-256
        # password = ???

        sha256_hash = hashlib.sha256()
        sha256_hash.update(password.encode('utf-8'))
        password = sha256_hash.hexdigest()

        # Connect to the database
        conn = get_db_connection()
        cursor = conn.cursor()

        # TODO # 2. Check if the user exists in the database and whether the password is correct
        # Query to check the user

        try:
            cursor.execute("SELECT password FROM users WHERE username = %s", (username,))
        except:
            print("crush")
            return "Username does not exist", 400

        result = cursor.fetchall() # fetchone() returns None if no record is found

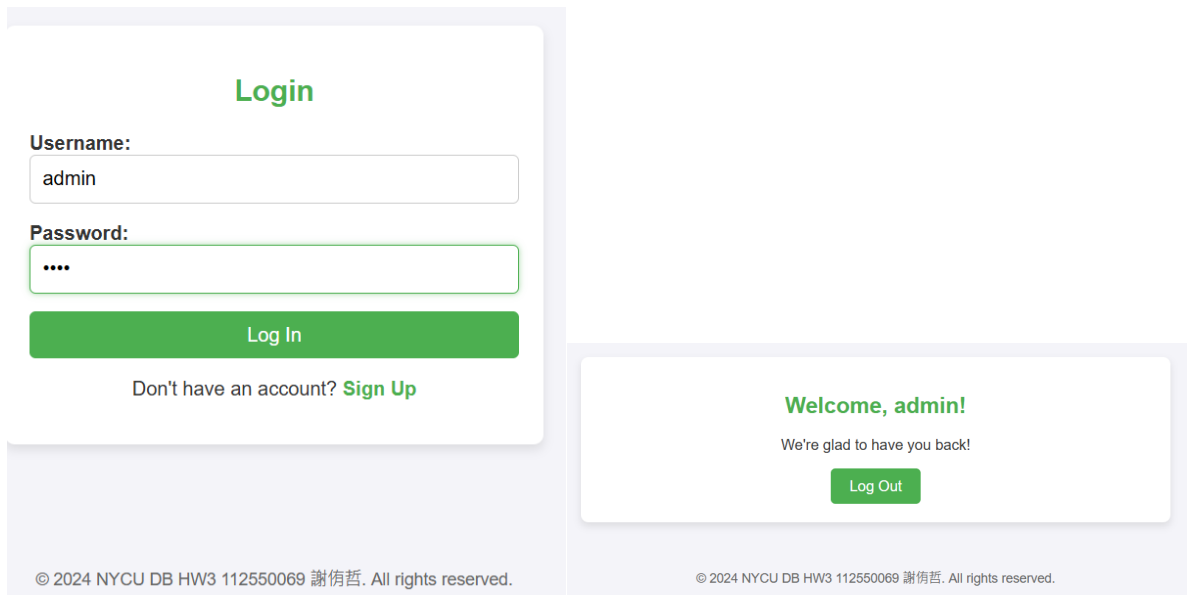
        result = cursor.fetchone() # fetchone() returns None if no record is found
        # if ???:
        cursor.close()
        conn.close()
        if result == None:
            flash("Username does not exist.", "danger")
            # return "Username does not exist", 400
        elif result[0] == password:
            session['username'] = username
            return redirect("/welcome")
        else:
            flash("Password is wrong.", "danger")
            # return "Password is wrong", 400

        # if pass the check, redirect to the welcome page and store the username in the session
        # session['username'] = username
        # return redirect("/welcome") # commit this line after completing TODO # 2

    return render_template("login.html")

```

首先用 router 接收根目錄的 request，如果是 post 的話，就讀入 post 的 body 並拿到裡面的 username 跟 password，接著把 password 做 sha256 的 hash，再開啟跟資料庫的 connection，使用 SQL 拿到符合使用者名稱的 password hash 值，如果使用者不存在，則回傳使用者不存在的 flash message，比對 password hash 值是否一樣，一樣就表示密碼正確，回傳對應 username 的 welcome 畫面，如果不同就回傳密碼錯誤的 flash message。



3. Screenshot and Explanation of Signup Function

```
# Signup
@app.route("/signup", methods=["GET", "POST"])
def signup():
    if request.method == "POST":
        username = request.form['username']
        password = request.form['password']

        # TODO # 4: Hash the password using SHA-256
        # password = ???
        sha256_hash = hashlib.sha256()
        sha256_hash.update(password.encode('utf-8'))
        password = sha256_hash.hexdigest()

        # Connect to the database
        conn = get_db_connection()
        cursor = conn.cursor()

        # TODO # 3: Add the query to insert a new user into the database
        try:
            cursor.execute("INSERT INTO users (username, password) \
                           VALUES (%s, %s);", (username, password))
            conn.commit()
            flash("Account created successfully! Please log in.", "success")
            return redirect("/")
        except mysql.connector.Error as err:
            flash(f"Error: {err}", "danger")
        finally:
            cursor.close()
            conn.close()

    return render_template("signup.html")
```

首先用 router 接收來自/signup 的 request，當他是 POST 的時候，就讀取 POST 帶的 body，把其中的 username 跟 password 讀進來，接著把 password 用 sha256 做 hash 後，開啟跟資料庫的 connection，用 SQL 把 username 跟 hash 後的 password 插入資料庫中，並使用參數化的方法避免 SQL injection，最後把 SQL commit 到資料庫讓他發揮效用。成功的話就輸出成功的 flash message，失敗則輸出失敗的 flash message，最後關掉 connection。(如果失敗，重新渲染 signup page)。

Success case:

The image displays two screenshots of a web application interface. The top screenshot shows the 'Sign Up' page. It has a title 'Sign Up' in green. Below it are two input fields: 'Username:' with the value 'guest' and 'Password:' with three dots indicating a password. A green 'Sign Up' button is below the fields. At the bottom, there is a link 'Already have an account? [Back to Login](#)'. The bottom screenshot shows the 'Login' page. It has a title 'Login' in green. Below it are two input fields: 'Username:' with the placeholder 'Enter your username' and 'Password:' with the placeholder 'Enter your password'. A green 'Log In' button is below the fields. At the bottom, there is a link 'Don't have an account? [Sign Up](#)'. A red flash message box at the bottom of the login page reads: '• Account created successfully! Please log in.'

© 2024 NYCU DB HW3 112550069 謝侑哲. All rights reserved.

Welcome, guest!

We're glad to have you back!

Log Out

© 2024 NYCU DB HW3 112550069 謝侑哲. All rights reserved.

Fail case:

Sign Up

Username:

admin

Password:

....

Sign Up

Already have an account? [Back to Login](#)

- Error: 1062 (23000): Duplicate entry 'admin' for key 'users.username'

Sign Up

Username:

Enter a username

Password:

Enter a password

Sign Up

Already have an account? [Back to Login](#)

- Error: 1062 (23000): Duplicate entry 'admin' for key 'users.username'

4. Screenshot of Hashed Password

```
sha256_hash = hashlib.sha256()  
sha256_hash.update(password.encode('utf-8'))  
password = sha256_hash.hexdigest()
```

首先創建一個 hashlib 的 hash object，然後把 password 以 utf-8 的形式 encode 並輸入給 object，最後拿出 object 算出來的對應十六進制值的字串。

```
mysql> select * from users  
-> ;  
+----+-----+-----+  
| id | username | password |  
+----+-----+-----+  
| 1 | admin | 03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4 |  
| 2 | youzhe | 4bc36f803d46036a145f58d10429b2623e78c25024a176a648cd7d79fd1b4591 |  
| 4 | root | 03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4 |  
+----+-----+-----+
```

5. Screenshot and Explanation of SQL Injection Prevention

The screenshot shows a web application login interface. The title is "Login" in green. There are two input fields: "Username:" and "Password:". The "Username:" field contains the text "admin' OR '1'='1", which is a common SQL injection payload. The "Password:" field contains a single dot ".". Below the input fields is a green "Log In" button. At the bottom, there is a link that says "Don't have an account? Sign Up".

Login

Username:

Password:

Log In

Don't have an account? [Sign Up](#)

- Username does not exist.

```
try:
    cursor.execute("SELECT password FROM users WHERE username = %s", (username,))
except:
    flash("Username does not exist.", "danger")
```

防禦 SQL injection 的方式就是利用參數化的方式，讓 SQL 認定我們傳入的參數是一個字串，就可以避免 SQL 改變到我們原本的 SQL，造成 SQL injection 的問題。